

Modified Training for SAVI Inference

The current decoupled training objective was designed without SAVI inference in mind. When SAVI is added at inference time, three mismatches arise between what the model was trained on and what it is asked to do. Each mismatch has a concrete fix; they can be applied incrementally.

Mismatch 1. Training input is full text history; SAVI inference input is canonical state prompt

The problem.

During training, the model sees [problem statement] + [full text history H_{t-1}] and generates x_t . During SAVI inference with canonical prompting, the model sees $f(S_{t-1}, P)$ and generates x_t . These are different inputs. The model has never seen the canonical prompt format during training, so there is a distributional shift at the input level that degrades performance even before any beam search or merging happens.

The fix.

Add training examples in canonical prompt format: replace the full text history with the corresponding canonical state representation $f(S_{t-1}, P)$, keeping all other loss terms unchanged. This is the smallest possible change with the largest expected return. L_{shape} , L_{commit} , and L_{flu} all apply unchanged on the canonical prompt format; the only change is what goes into the context window.

Mismatch 2. Training follows one ground-truth path; SAVI inference explores many paths

The problem.

Teacher forcing means the model only ever sees ground-truth prefixes during training. In SAVI inference, the beam contains states reached via paths that are not ground truth but are semantically valid — the model has never been trained to continue coherently from these states. As beam width increases, a larger fraction of states the model encounters at inference time are off-trajectory, and performance degrades accordingly.

The fix.

During training, generate multiple candidate continuations from each state, run Φ to verify each one, and train on all valid (S, x, S') triples where $\Phi(S, x) \neq \perp$, not just the ground-truth transition. This teaches the model to continue coherently from any valid semantic state, not only the ground-truth state. Concretely: at each training step, sample N candidates, discard those where Φ returns $\perp$, and include the rest as additional training pairs with the same $L_{\text{shape}} + L_{\text{commit}}$ objective.

Mismatch 3. Training optimizes token-level likelihood; SAVI optimizes semantic-state-level path score

The problem.

L_commit trains the model to write a calibrated COMMIT: line, but the calibration target p^* is derived from the original task prompt. In SAVI inference, the model is given a canonical state prompt $f(S_{t-1}, P)$, and p^* at that state may differ. More fundamentally, token-level likelihood does not distinguish between candidates that lead to valid semantic states and those that produce $\perp$. A candidate can have high $\log P_{\text{fwd}}$ and still be discarded by Phi.

The fix.

Add a verifier signal: sample x , run $\text{Phi}(S, x)$, and add a penalty when Phi returns $\perp$ and a reward when Phi returns a valid state. This upgrades L_commit from *write out a correctly formatted line* to *write a line that Phi can parse into a valid semantic state*, directly aligning training with SAVI's objective. The signal is cheap to obtain: Phi is deterministic (SymPy or a code interpreter), so every sampled candidate can be verified without an additional model call.

Implementation order

Step 1. Add canonical prompt format to the training data. All existing loss terms apply unchanged. This can be done immediately with no new infrastructure.

Step 2. Train L_commit and the recovery term on canonical prompts specifically, so the model writes calibrated COMMIT: lines when given $f(S_{t-1}, P)$ as input, not only the original task prompt.

Step 3. Add the Phi verification reward: for each sampled candidate during training, run Phi and penalize $\perp$ returns. This reduces the non-commitment rate further and aligns the model with SAVI's beam validity requirement.

Step 4. If steps 1-3 produce pass@1 gains but leave a gap to the oracle ceiling, add sequence-level REINFORCE using SymPy or execution-based final-answer correctness as reward. This is the most expensive step and should only be attempted once the earlier steps are validated.

The key diagnostic after each step: measure coverage (how many distinct semantic states appear in the beam), non-commitment rate (fraction of candidates where Phi returns $\perp$), and pass@1 on GSM8K or a code benchmark. Step 1 should improve coverage. Step 3 should reduce the $\perp$ rate. Pass@1 improvement is the signal that the full pipeline is working.